
ASSIGNMENT NO:8

AIM: Mini Project (In group): A Distributed Application for Interactive Multiplayer Games

Objective: To create single deployable artifact to run game servers by containerizing the applications

Outcome: Each game is run as a separate process (dedicated server) to prevent impacting other game instances by resource usage

Explanation:

This is the main part of engineering, we're going to create a schematic model of our mental model for the whole system. to start we talk about the chosen stack to know the limitations and costs

Deployment and orchestration

We have a single deployable artifact to run game servers by containerizing the applications. But deploying, maintaining, and scaling these artifacts and services is hard work to do. The Kubernetes helps handle deployments, orchestrate running containers, and manage nodes.

Load Balancer

The game manager is scalable horizontally, which means that the k8s run multiple instances of it concurrently. Therefore, we need a load balancer to distribute the requests to the game manager pods. The Nginx ingress is used as a load balancer to handle incoming requests and proxy them to the services.

Game Client

I chose Unity3D for the game client. The Unity Game Engine uses C# as the programming script.

Game Server

The game server is an authoritative server, which means that each client input is processed and validated on the server-side, then the game client checks the server validated data with its predictions. In addition, each game is run as a separate process (dedicated server) to prevent impacting other game instances by resource usage or corruption, for example, if a game is crashed, other games remain safe to continue running. The game server is a stateful server because it uses memory to store game states like players' coordinations and inputs, so it's not replaceable or scalable horizontally. Our game is a fast-paced multiplayer game and should use UDP protocol to stream the game states and receive the players' inputs in the fastest way. Also, we need a TCP tunnel to transfer the game events which must be received with acknowledgment and in order. The game world simulations and calculations are not complicated, so I chose golang to develop the game server.

Game Manager Service

I merged some services with the game manager for the greater good (make deployment and maintenance easy), thus it must handle a lot of work. I chose golang to develop the game manager service. The game manager app is a stateless service and could be scaled horizontally.

APIs

- An HTTP server APIs to handle the client requests in the game menu and store.
- A WebSocket handler to keep a long-living connection with the client in the main menu to send events.

Matchmaking

The matchmaking service is responsible for putting the players in a queue and categorizing them depending on latency and rank. The Redis fits with these conditions. It supports the atomic lock concept which can be used for race condition challenges in multiple game manager instances.

Database

A database is needed to store some data like cars, items, users items, game logs, and so on. to achieve this, I used MySQL to store relational data (users' customized car parts), for the game data MongoDB fits here, but for simplicity, I moved forward with the MySQL.

Kubernetes API

The game manager uses K8S API to create a new pod and run the game server container as a dedicated server.

Game Session Management

The session manager creates, caches, manages, and destroys the game sessions.

Event Broker

The services talk to each other using the event broker, like when two players are matched or the game server got ready. I've looked for an opportunity to test the KubeMQ and this situation seemed good for me to use it. I used the KubeMQ community version for this. Also, Kafka was a good option, but on this scale, I prefer not to move forward with it for this project because of RAM consumption and node resource limits to decrease the server costs.

Result:

Distributed Application for Interactive Multiplayer Games

Date:	
Marks obtained:	
Sign of course coordinator:	
Name of course Coordinator :	